Nama : Khania Puji Auliya

NIM : 254107020236

Kelas : 1G

Prodi : D-IV Teknik Informatika

LAPORAN JOBSHEET 5

Percobaan 5.2

- Kode Program Faktorial15.java

```
package BruteForceDivideConquer.minggu5;
public class Faktorial15 {
    int faktorialBF(int n) {
        int fakto = 1;
        for (int i = 1; i <= n; i++) {
            fakto = fakto * i;
        }
        return fakto;
    }
    int faktorialDC(int n) {
        if (n == 1) {
            return 1;
        } else {
            int fakto = n * faktorialDC(n - 1);
            return fakto;
        }
    }
}
```

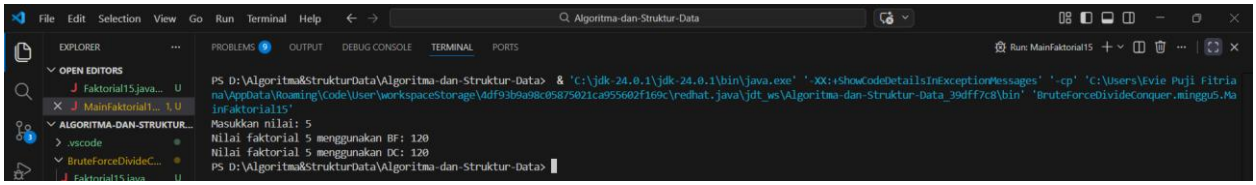
- Kode Program Faktorial15.java

```
package BruteForceDivideConquer.minggu5;
import java.util.Scanner;
```

```
public class MainFaktorial15 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Masukkan nilai: ");
        int nilai = input.nextInt();

        Faktorial15 fk = new Faktorial15();
        System.out.println("Nilai faktorial " + nilai + " menggunakan BF: " +
fk.faktorialBF(nilai));
        System.out.println("Nilai faktorial " + nilai + " menggunakan DC: " +
fk.faktorialDC(nilai));
    }
}
```

- Hasil Running



- Pertanyaan
  1. Pada base line Algoritma Divide Conquer untuk melakukan pencarian nilai faktorial, jelaskan perbedaan bagian kode pada penggunaan if dan else!
  2. Apakah memungkinkan perulangan pada method faktorialBF() diubah selain menggunakan for? Buktikan!
  3. Jelaskan perbedaan antara fakto *= i; dan int fakto = n * faktorialDC(n-1); !
  4. Buat Kesimpulan tentang perbedaan cara kerja method faktorialBF() dan faktorialDC()!

- Jawaban
  1. Pada algoritma Divide and Conquer, bagian if berfungsi sebagai base case atau kondisi dasar agar proses rekursi berhenti. Sedangkan bagian else digunakan untuk melakukan

proses rekursif, yaitu memanggil kembali method faktorialDC() dengan nilai n-1 sampai mencapai kondisi dasar.

2. Ya, perulangan pada method faktorialBF() memungkinkan diubah selain menggunakan for, misalnya menggunakan while.

   Contoh:

```
int faktorialBF(int n) {
    int fakto = 1;
    int i = 1;
    while (i <= n) {
        fakto = fakto * i;
        i++;
    }
    return fakto;
}
```

3. Pernyataan fakto *= i; digunakan pada metode Brute Force dengan cara melakukan perkalian secara berulang dalam perulangan sampai mencapai nilai n. Sedangkan int fakto = n * faktorialDC(n-1); digunakan pada metode Divide and Conquer, yaitu menghitung faktorial dengan cara rekursif dengan memanggil kembali method yang sama hingga mencapai kondisi dasar.
4. method faktorialBF() menghitung nilai faktorial menggunakan perulangan (iterasi) dari 1 sampai n sehingga prosesnya berjalan secara berurutan. Sedangkan method faktorialDC() menggunakan rekursi dengan pendekatan Divide and Conquer, yaitu memecah masalah menjadi submasalah yang lebih kecil dengan memanggil fungsi yang sama hingga mencapai kondisi dasar.

Percobaan 5.3

- Kode Program Pangkat15.java

```
package BruteForceDivideConquer.minggu5;
public class Pangkat15 {
    int nilai, pangkat;
```

```
    Pangkat15(int n, int p) {
        nilai = n;
        pangkat = p;
    }
    int pangkatBF(int a, int n) {
        int hasil = 1;
        for (int i = 0; i < n; i++) {
            hasil = hasil * a;
        }
        return hasil;
    }
    int pangkatDC(int a, int n) {
        if (n == 1) {
            return a;
        } else {
            if (n % 2 == 1) {
                return (pangkatDC(a, n / 2) * pangkatDC(a, n / 2) * a);
            } else {
                return (pangkatDC(a, n / 2) * pangkatDC(a, n / 2));
            }
        }
    }
}
```

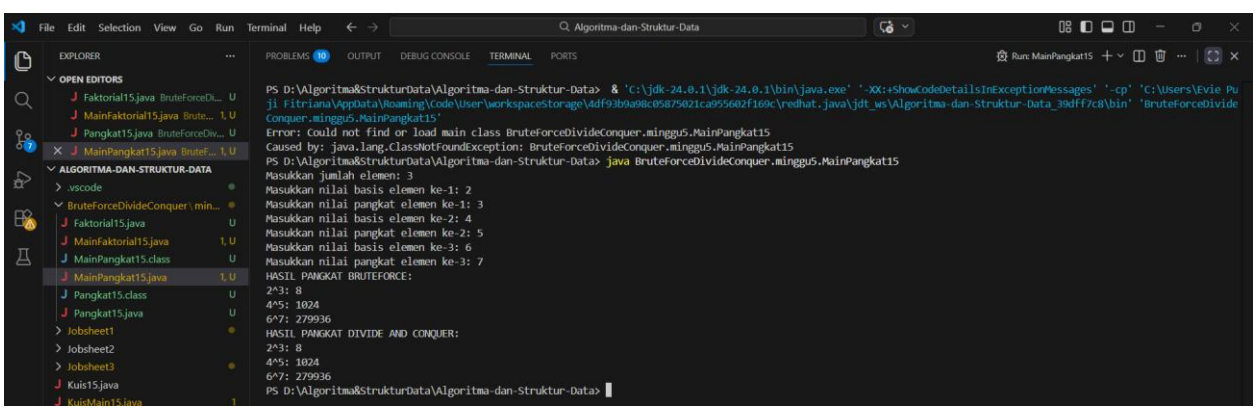
- Kode Program MainPangkat15.java

```
package BruteForceDivideConquer.minggu5;
import java.util.Scanner;
public class MainPangkat15 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Masukkan jumlah elemen: ");
```

```
        int elemen = input.nextInt();

        Pangkat15[] png = new Pangkat15[elemen];
        for (int i = 0; i < elemen; i++) {
            System.out.print("Masukkan nilai basis elemen ke-" + (i + 1) + ": ");
            int basis = input.nextInt();
            System.out.print("Masukkan nilai pangkat elemen ke-" + (i + 1) + ": ");
            int pangkat = input.nextInt();
            png[i] = new Pangkat15(basis, pangkat);
        }
        System.out.println("HASIL PANGKAT BRUTEFORCE: ");
        for (Pangkat15 p : png) {
            System.out.println(p.nilai + "^" + p.pangkat + ": " + p.pangkatBF(p.nilai, p.pangkat));
        }
        System.out.println("HASIL PANGKAT DIVIDE AND CONQUER: ");
        for (Pangkat15 p : png) {
            System.out.println(p.nilai + "^" + p.pangkat + ": " + p.pangkatDC(p.nilai, p.pangkat));
        }
    }
}
```

- Hasil Running



- Pertanyaan
  1. Jelaskan mengenai perbedaan 2 method yang dibuat yaitu pangkatBF() dan pangkatDC()!
  2. Apakah tahap combine sudah termasuk dalam kode tersebut?Tunjukkan!
  3. Pada method pangkatBF()terdapat parameter untuk melewatkan nilai yang akan dipangkatkan dan pangkat berapa, padahal di sisi lain di class Pangkat telah ada atribut nilai dan pangkat, apakah menurut Anda method tersebut tetap relevan untuk memiliki parameter? Apakah bisa jika method tersebut dibuat dengan tanpa parameter? Jika bisa, seperti apa method pangkatBF() yang tanpa parameter?
  4. Tarik tentang cara kerja method pangkatBF() dan pangkatDC()!

- Jawaban
  1. Method pangkatBF() menggunakan perulangan (iterasi) untuk mengalikan nilai basis sebanyak nilai pangkat. Sedangkan pangkatDC() menggunakan rekursi dengan metode Divide and Conquer, yaitu memecah perhitungan menjadi pangkat yang lebih kecil lalu menggabungkan hasilnya.
  2. Ya, tahap **combine** ada pada bagian:
     pangkatDC(a, n / 2) * pangkatDC(a, n / 2)
     pangkatDC(a, n / 2) * pangkatDC(a, n / 2) * a
  3. Method tetap bisa dibuat tanpa parameter karena sudah ada atribut nilai dan pangkat. Contoh:

```
int pangkatBF() {
    int hasil = 1;
    for (int i = 0; i < pangkat; i++) {
        hasil *= nilai;
    }
    return hasil;
}
```

  4. pangkatBF() menghitung pangkat dengan perulangan, sedangkan pangkatDC() menghitung pangkat dengan rekursi (Divide and Conquer) dengan memecah masalah menjadi lebih kecil.

Percobaan 5.4

- Kode Program Sum15.java

```
package BruteForceDivideConquer.minggu5;
public class Sum15 {
    double keuntungan[];
    Sum15(int el) {
        keuntungan = new double[el];
    }
    double totalBF() {
        double total = 0;
        for (int i = 0; i < keuntungan.length; i++) {
            total = total + keuntungan[i];
        }
        return total;
    }
    double totalDC(double arr[], int l, int r) {
        if (l == r) {
            return arr[l];
        }
        int mid = (l + r) / 2;
        double lsum = totalDC(arr, l, mid);
        double rsum = totalDC(arr, mid + 1, r);
        return lsum + rsum;
    }
}
```

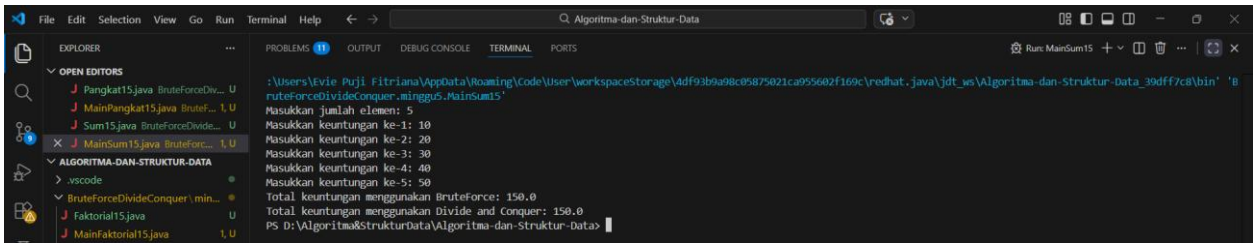
- Kode Program MainSum15.java

```
package BruteForceDivideConquer.minggu5;
import java.util.Scanner;
public class MainSum15 {
    public static void main(String[] args) {
```

```
        Scanner input = new Scanner(System.in);
        System.out.print("Masukkan jumlah elemen: ");
        int elemen = input.nextInt();

        Sum15 sm = new Sum15(elemen);
        for (int i = 0; i < elemen; i++) {
            System.out.print("Masukkan keuntungan ke-" + (i + 1) + ": ");
            sm.keuntungan[i] = input.nextDouble();
        }
        System.out.println("Total keuntungan menggunakan BruteForce: " + sm.totalBF());
        System.out.println(
                "Total keuntungan menggunakan Divide and Conquer: " +
sm.totalDC(sm.keuntungan, 0, elemen - 1));
    }
}
```

- Hasil Running



- Pertanyaan
  1. Kenapa dibutuhkan variable mid pada method TotalDC()?
  2. Untuk apakah statement di bawah ini dilakukan dalam TotalDC()?

     ```
     double lsum = totalDC(arr, l, mid);
     double rsum = totalDC(arr, mid+1, r);
     ```

  3. Kenapa diperlukan penjumlahan hasil lsum dan rsum seperti di bawah ini?

     ```
     return lsum+rsum;
     ```

  4. Apakah base case dari totalDC()?
  5. Tarik Kesimpulan tentang cara kerja totalDC()

- Jawaban

1. Variabel mid digunakan untuk menentukan titik tengah array agar data dapat dibagi menjadi dua bagian dalam metode Divide and Conquer.
2. Fungsi tersebut digunakan untuk menghitung jumlah elemen pada bagian kiri dan kanan array secara rekursif.
3. Penjumlahan lsum + rsum dilakukan untuk menggabungkan hasil dari bagian kiri dan kanan sehingga didapat total keseluruhan.
4. Base Case nya yaitu: if (l == r), yaitu saat array hanya memiliki satu elemen.
5. totalDC() bekerja dengan membagi array menjadi dua bagian, menghitung masing-masing bagian secara rekursif, lalu menjumlahkan hasilnya untuk mendapatkan total.